



Simulador de Tienda de Mascotas Virtual

Profesor

Geoffrey Jean-Pierre Christophe Hecht

Integrantes

Bastian Ignacio Molina Contreras

Isaac Amadeus Nelson Castro Villalobos

Martin Ignacio Carrasco Perez

Introducción

El presente trabajo ofrece al usuario la oportunidad de administrar una tienda de mascota, donde este puede adquirir distintos tipos de mascotas (perros, gatos, peces y pájaros), proporcionarles cuidado y atención de forma activa como alimentarlas, limpiar sus habitats, jugar con ellas y tratar su salud.

El jugador también tiene la opción de vender las mascotas a clientes virtuales que llegaran a la tienda buscando por una mascota de un tipo en específico.

Esta tienda virtual fue desarrollada en Java, utilizando Swing para la interfaz gráfica (Frontend) y aplicando los patrones de diseño State, Observer y Template Method.

Descripción

El sistema se organiza en dos capas principales:

- **Backend:** Contiene toda la lógica de la tienda virtual. Contiene las clases de las mascotas, los estados de las mascotas, la tienda, los suministros y el cliente virtual.

- **Frontend:** Contiene las tres principales ventanas utilizando Swing que permite al usuario interactuar con la logica de la tienda.

Al iniciar la aplicación, el jugador accede a la primera ventana (V1_inicio) donde tiene dos opciones:

- **Gestionar tienda (V2_tienda):** Permite al usuario comprar mascotas y suministros, revisar el inventario de la tienda y vender mascotas.
- **Mascotas (V3_mascotas):** Permite ver y cuidar a cada mascota de forma individual. Despliega el nombre, estado y barras de progreso para el hambre, felicidad, higiene, y salud. El jugador puede decidir si quiere alimentarlas, limpiarlas, jugar con ellas y atender su salud.

Decisiones

Durante el desarrollo de este proyecto se tomaron múltiples decisiones respecto al diseño:

- El inventario de la tienda fue integrado en la sección de gestión de tienda (V2_tienda), debido a que resulta lógica y es más cómodo y intuitivo para el usuario tener la capacidad de administrar todo desde el un único lugar.

- La clase perro incluye el atributo raza, lo cual aporta un mayor realismo a la identidad de esta mascota.
- La degradación de atributos se implementó mediante un timer de 3 segundos en V3_mascotas el cual llama al método pasarTiempo() de cada mascota, lo cual simula el paso del tiempo de forma automática mientras el jugador cuida de sus mascotas.
- El método atenderSalud() solo funciona si la mascota ha perdido un poco de salud, y consume un medicamento del inventario, lo cual vincula esta mecánica de cuidados con la gestión de suministros.
- Para alimentar a una mascota, se necesita tener comida en el inventario, lo que fomenta que el usuario este constantemente pendiente de sus suministros.

Patrones de diseño

State

Se utilizo el patron State para modelar el estado de la mascota segun su salud y bienestar. Cada estado aplica una penalización distinta sobre los atributos de la mascota al pasar el tiempo.

Los estados son:

- EstadoSaludable: No aplica penalizaciones.
- EstadoTriste: Reduce la salud por 1 punto por ciclo.
- EstadoEnfermo: Reduce la salud y felicidad 3 puntos por ciclo
- EstadoCritico, reduce la salud en 7 puntos, la felicidad en 8 puntos y el hambre en 5 puntos por ciclo.

El cambio entre estos estados ocurre automáticamente en el método ActualizarEstado() de Mascota, se rige por los atributos salud, hambre y felicidad.

Observer

Se implemento el patrón Observer para que la interfaz gráfica se actualice de forma automática cada vez que los atributos de una mascota cambian.

Cada vez que se llama pasarTiempo(), alimentar(), limpiar(), jugar() o atenderSalud(), la mascota notifica a todos sus observers. Lo que permite que las barras de progreso de los atributos de la mascota este constantemente actualizándose.

Template Method

Se utilizo este patron para la creacion de el algoritmo de degradacion de atributos en la clase abstracta Mascota, permitiendo que cada subclase defina sus propios valores especificos.

El método pasarTiempo() en mascota define los pasos que ocurren cada vez que pasa el tiempo, bajar el hambre, felicidad, higiene, actualizar el estado de la mascota, etc. Estos pasos son siempre los mismos para cualquier mascota.

Lo que cambia entre las mascotas es que tanto se degrada cada atributo. Para eso, cada subclase implementa tres métodos:

- getDegradacionHambre()
- getDegradacionFelicidad()
- getDegradacionHigiene()

Donde se definen sus propios valores.

Autocritica

Naturalmente el proyecto tiene varios aspectos de mejora y funcionalidades que no lograros ser implementadas tales como:

- Distintos tipos de alimentos según el tipo de mascota que aumente el realismo.
- Distintos tipos de medicamentos según el estado de salud de la mascota, lo que permitiría agregar múltiples tipos de enfermedad.
- Compra de juguetes para que las mascotas aumenten su felicidad por si sola por un periodo de tiempo.
- Una interfaz mas visual y llamativa del inventario, donde se desplieguen imagenes de los suministros.

Hablando de las dificultades, el mayor desafío diríamos que fue implementar el frontend, ya que aquí se encuentra nuestra mayor debilidad, además que para ciertas cosas como por ejemplo la degradación natural de los atributos de las mascotas, se debe tener relación entre los integrantes encargados del frontend y el backend.

A pesar de todos nuestros desafíos y dificultades, logramos sobrellevar de buena forma el proyecto, dividiendo en secciones bien definidas este, y logrando una implementacion de las funcionalidades principales del simulador.